lpc

April 19, 2026

[1]:
```
import numpy as np
import scipy.io.wavfile as wav
import matplotlib.pyplot as plt
from IPython.display import display, Audio
```

1 LP Analysis

[2]:
```
from scipy.linalg import solve_toeplitz
from scipy.signal import lfilter, lfiltic
```

[3]:
```
def windowing(signal, frame_length_ms, sampling_rate):
    """Windowing signal with no overlap"""
    frame_length = int(sampling_rate * frame_length_ms / 1000)
    window = np.hamming(frame_length)
    number_of_frames = int(np.ceil(len(signal) / frame_length))
    pad_amount = number_of_frames * frame_length - len(signal)
    padded_signal = np.pad(signal, ((0, pad_amount)))

    frame_matrix_orig = np.zeros((number_of_frames, frame_length))
    frame_matrix_windowed = np.zeros((number_of_frames, frame_length))
    for i in range(0, number_of_frames,):
        start_index = i * frame_length
        end_index = start_index + frame_length
        frame_matrix_orig[i, :] = padded_signal[start_index:end_index]
        frame_matrix_windowed[i, :] = np.multiply(padded_signal[start_index:
 ↪end_index], window)

    return frame_matrix_orig, frame_matrix_windowed
```

[4]:
```
def autocorrelation(frame, p):
    result = np.correlate(frame, frame, mode='full')
    mid = len(result) // 2
    result = result[mid:mid + p + 1]
    return result
```

[5]:
```
def solve_lpc_coefficient(corr_coef):
    # Construct toeplitz matrix
    corr_matrix = np.zeros((len(corr_coef) - 1, len(corr_coef) - 1))

    for i in range(corr_matrix.shape[0]):
        for j in range(corr_matrix.shape[1]):
            corr_matrix[i, j] = corr_coef[abs(i - j)]

    lpc_coef = solve_toeplitz((corr_matrix[0, :], corr_matrix[:, 0]),␣
 ↪corr_coef[1:])
    # lpc_coef = np.linalg.solve(corr_matrix, corr_coef[1:])
    return lpc_coef
```

[6]:
```
def lpc_residual_frame(frame, lpc_coef, prev_samples):
    A = np.concatenate(([1.0], -lpc_coef))
    extended_input = np.concatenate((prev_samples, frame))
    residual_extended = lfilter(A, [1.0], extended_input)
    # Discard the first p samples of the output (corresponding to prev_samples)
    residual = residual_extended[len(prev_samples):]

    return residual
```

[7]:
```
```

```python
def lpc_analysis(signal, fs, p, frame_length_ms):
    """
    Perform LPC analysis on an entire signal (no frame overlap).

    Parameters
    ----------
    signal : 1D ndarray
        Input speech signal.
    fs : int
        Sampling frequency in Hz.
    p : int
        LPC order.
    frame_length_ms : float
        Frame length in milliseconds.

    Returns
    -------
    residual_frames : 2D ndarray [num_frames, frame_length]
        LPC residual per frame.
    lpc_coeffs : 2D ndarray [num_frames, p]
        LPC coefficients per frame.
    """
    frame_matrix_orig, frame_matrix_windowed = windowing(signal,␣
↪frame_length_ms, fs)
    num_frames, _ = frame_matrix_windowed.shape

    residual_frames = np.zeros_like(frame_matrix_orig)
    lpc_coeffs = np.zeros((num_frames, p))

    # Initial previous samples: zeros for the very first frame
    prev_samples = np.zeros(p)

    for i in range(num_frames):
        win_frame = frame_matrix_windowed[i, :]
        orig_frame = frame_matrix_orig[i, :]
        r = autocorrelation(win_frame, p)
        a = solve_lpc_coefficient(r)
        lpc_coeffs[i, :] = a

        residual = lpc_residual_frame(orig_frame, a, prev_samples)
        residual_frames[i, :] = residual

        # Update previous samples for next frame (non-windowed)
        prev_samples = orig_frame[-p:]

    return residual_frames, lpc_coeffs
```

[8]:
```
fs, male_speech = wav.read('../data/Male_speech/si745_8kHz.wav')
male_speech = male_speech / max(abs(male_speech))
display(Audio(male_speech, rate=fs))

residual_frames_male, lpc_coeffs_male = lpc_analysis(male_speech, fs, p=10,␣
 ↪frame_length_ms=25)
residual_full_male = residual_frames_male.flatten()
residual_full_male = residual_full_male[:len(male_speech)]
t = np.arange(len(male_speech)) / fs

plt.figure(figsize=(6, 4))
plt.plot(t, male_speech, label='Original speech')
plt.plot(t, residual_full_male, label='LPC residual', alpha=0.8)
plt.xlabel('Time [s]')
plt.ylabel('Amplitude')
#plt.title('Original Speech Signal and LPC Residual of si745_8kHz.wav')
plt.legend(loc='upper right')
plt.grid(True)
plt.tight_layout()
#plt.show()
plt.savefig('../fig/si745_analysis.png')
plt.close()
```

```
<IPython.lib.display.Audio object>
```

```
<img src="../fig/si745_analysis.png" alt="LPC Analysis Plot" style="width:650px; height:auto;"
<p>
    <b>Figure 1.</b>Original speech waveform and LPC residual for the male speech signal
    (si745_8kHz.wav). The residual represents the excitation signal obtained after
    removing the linear predictive spectral envelope using a 10th-order LPC model
    with 25 ms frames.
</p>
```

[9]:
```
```

```
fs, female_speech = wav.read('../data/Female_speech/si747_8kHz.wav')
female_speech = female_speech / max(abs(female_speech))
display(Audio(female_speech, rate=fs))

residual_frames_female, lpc_coeffs_female = lpc_analysis(female_speech, fs,␣
 ↪p=10, frame_length_ms=25)
residual_full_female = residual_frames_female.flatten()
residual_full_female = residual_full_female[:len(female_speech)]
t = np.arange(len(female_speech)) / fs

plt.figure(figsize=(6, 4))
plt.plot(t, female_speech, label='Original speech')
plt.plot(t, residual_full_female, label='LPC residual', alpha=0.8)
plt.xlabel('Time [s]')
plt.ylabel('Amplitude')
#plt.title('Original Speech Signal and LPC Residual of si747_8kHz.wav')
plt.legend(loc='upper right')
plt.grid(True)
plt.tight_layout()
#plt.show()
plt.savefig('../fig/si747_analysis.png')
plt.close()
```

```
<IPython.lib.display.Audio object>
```

```
<img src="../fig/si747_analysis.png" alt="LPC Analysis Plot" style="width:650px; height:auto;"
<p>
    <b>Figure 2.</b>Original speech waveform and LPC residual for the female speech signal
    (si747_8kHz.wav). The residual represents the excitation signal obtained after
    removing the linear predictive spectral envelope using a 10th-order LPC model
    with 25 ms frames.
</p>
```

2 LPC synthesis

[10]:

```python
def lpc_synthesis(residual_frames, lpc_coeffs):
    """
    Perform LPC synthesis to reconstruct signal from residual.

    Parameters
    ----------
    residual_frames : 2D ndarray [num_frames, frame_length]
        LPC residual per frame from analysis stage.
    lpc_coeffs : 2D ndarray [num_frames, p]
        LPC coefficients per frame.

    Returns
    -------
    reconstructed_signal : 1D ndarray
        Reconstructed speech signal.
    reconstructed_frames : 2D ndarray [num_frames, frame_length]
        Reconstructed signal organized by frames.
    """
    num_frames, frame_length = residual_frames.shape
    p = lpc_coeffs.shape[1]

    reconstructed_frames = np.zeros_like(residual_frames)

    # Initialize previous reconstructed samples
    prev_reconstructed_samples = np.zeros(p)

    for i in range(num_frames):
        a = lpc_coeffs[i, :]
        b = np.array([1.0])
        A = np.concatenate(([1.0], -a))
        residual = residual_frames[i, :]

        # Compute initial conditions from previous reconstructed samples␣
↪(y[n-1], ..., y[n-p])
        # lfiltic expects: b, a, y_history (reversed)
        y_hist_reversed = prev_reconstructed_samples[::-1]
        zi = lfiltic(b, A, y_hist_reversed)

        reconstructed_frame, _ = lfilter(b, A, residual, zi=zi)
        reconstructed_frames[i, :] = reconstructed_frame

        prev_reconstructed_samples = reconstructed_frame[-p:]

    reconstructed_signal = reconstructed_frames.flatten()
    return reconstructed_signal, reconstructed_frames
```

[11]:
```
reconstruted_signal_male, reconstructed_frames_male =␣
 ↪lpc_synthesis(residual_frames_male, lpc_coeffs_male)
display(Audio(reconstruted_signal_male, rate=fs))

t = np.arange(len(male_speech)) / fs
plt.figure(figsize=(6, 4))
plt.plot(t, male_speech, label='Original speech')
plt.plot(t, reconstruted_signal_male[:len(male_speech)], label='Reconstructed␣
 ↪speech', alpha=0.8)
plt.xlabel('Time [s]')
plt.ylabel('Amplitude')
#plt.title('Original Speech Signal and Reconstructed Speech si745_8kHz.wav')
plt.legend(loc='upper right')
plt.grid(True)
plt.tight_layout()
#plt.show()
plt.savefig('../fig/si745_reconstruction.png')
plt.close()
```

```
<IPython.lib.display.Audio object>
```

```
<img src="../fig/si745_reconstruction.png" alt="LPC Analysis Plot" style="width:650px; height:a
<p>
    <b>Figure 3.</b> Comparison between the original male speech signal and the reconstructed
    speech obtained through LPC synthesis. The reconstruction uses the LPC residual
    and the estimated LPC coefficients to recreate the speech waveform.
</p>
```

[12]:
```
reconstructed_signal_female, reconstructed_frames_female =␣
 ↪lpc_synthesis(residual_frames_female, lpc_coeffs_female)
display(Audio(reconstructed_signal_female, rate=fs))

t = np.arange(len(female_speech)) / fs
plt.figure(figsize=(6, 4))
plt.plot(t, female_speech, label='Original speech')
plt.plot(t, reconstructed_signal_female[:len(female_speech)],␣
 ↪label='Reconstructed speech', alpha=0.8)
plt.xlabel('Time [s]')
plt.ylabel('Amplitude')
#plt.title('Original Speech Signal and Reconstructed Speech si747_8kHz.wav')
plt.legend(loc='upper right')
plt.grid(True)
plt.tight_layout()
#plt.show()
plt.savefig('../fig/si747_reconstruction.png')
plt.close()
```

```
<IPython.lib.display.Audio object>
```

```
<img src="../fig/si747_reconstruction.png" alt="LPC Analysis Plot" style="width:650px; height:
<p>
    <b>Figure 4.</b> Comparison between the original female speech signal and the reconstructe
    speech obtained through LPC synthesis. The reconstruction uses the LPC residual
    and the estimated LPC coefficients to recreate the speech waveform.
</p>
```

3 Quantize

[13]:
```
def lpc_to_reflection_coef(lpc_coef):
    p = len(lpc_coef)
    k = np.zeros(p)
    a = np.copy(lpc_coef)

    for i in range(p-1, -1, -1):
        k[i] = a[i]
        a_new = np.zeros(p)

        for m in range(0, i):
            a_new[m] = (a[m] + k[i] * a[i-m-1]) / (1 - k[i]**2)

        a = a_new
    return k
```

[14]:
```
def reflection_to_lpc_coef(reflection_coef):
    p = len(reflection_coef)
    a = np.zeros(p)
    k = np.copy(reflection_coef)

    for i in range(p):
        a_new = np.zeros(p)
        a_new[i] = k[i]
        for m in range(i):
            a_new[m] = a[m] - k[i] * a[i-1-m]

        a = a_new

    return a
```

[15]:
```

```

```
def quantize_reflection_coefficients(reflection_coef, num_bits):
    num_levels = 2**num_bits
    reflection_coef_max = 1
    reflection_coef_min = -1

    # Quantization step size
    delta = (reflection_coef_max - reflection_coef_min) / num_levels

    # Quantize: map to nearest quantization level
    k_indices = np.floor((reflection_coef - reflection_coef_min) / delta)
    k_quantized = (k_indices + 0.5) * delta + reflection_coef_min

    return k_quantized
```

[16]:
```
from scipy.signal import freqz

def compute_spectral_distortion(lpc_unquant, lpc_quant, n_fft=1024):
    A_unquant = np.concatenate(([1.0], -lpc_unquant))
    A_quant = np.concatenate(([1.0], -lpc_quant))

    _, H_unquant = freqz([1.0], A_unquant, worN=n_fft, whole=False)
    _, H_quant = freqz([1.0], A_quant, worN=n_fft, whole=False)

    # Power Spectra
    P_unquant = np.abs(H_unquant)**2
    P_quant = np.abs(H_quant)**2

    log_P_unquant = 10 * np.log10(P_unquant)
    log_P_quant = 10 * np.log10(P_quant)

    squared_diff = (log_P_unquant - log_P_quant)**2
    sd_squared = 1 / (n_fft / 2 + 1) * np.sum(squared_diff[:n_fft // 2 + 1])
    sd = np.sqrt(sd_squared)

    return sd
```

[17]:
```

```

```
def compute_average_sd_for_signal(lpc_coeffs_unquant, lpc_coeffs_quant,␣
 ↪n_fft=1024):
    num_frames = lpc_coeffs_unquant.shape[0]
    sd_per_frame = np.zeros(num_frames)

    for i in range(num_frames):
        sd_per_frame[i] = compute_spectral_distortion(
            lpc_coeffs_unquant[i, :],
            lpc_coeffs_quant[i, :],
            n_fft
        )

    avg_sd = np.mean(sd_per_frame)

    return avg_sd
```

[18]:
```
def compute_sd_for_quantization_schemes(lpc_coeffs, num_bits_list=[5, 4, 3]):
    results = {}
    quantized_lpc = {}

    for num_bits in num_bits_list:
        num_frames, _ = lpc_coeffs.shape
        lpc_coeffs_quant = np.zeros_like(lpc_coeffs)

        for i in range(num_frames):
            reflection_coef = lpc_to_reflection_coef(lpc_coeffs[i, :])
            reflection_coef_quant =␣
 ↪quantize_reflection_coefficients(reflection_coef, num_bits)
            lpc_coeffs_quant[i, :] =␣
 ↪reflection_to_lpc_coef(reflection_coef_quant)

        avg_sd = compute_average_sd_for_signal(lpc_coeffs, lpc_coeffs_quant,␣
 ↪n_fft=1024)

        results[num_bits] = avg_sd
        quantized_lpc[num_bits] = lpc_coeffs_quant

    return results, quantized_lpc
```

[19]:
```

```

```
num_bits_list = [5, 4, 3]
print("Average SD for si745_8kHz.wav")
results, quantized_lpc = compute_sd_for_quantization_schemes(lpc_coeffs_male,␣
 ↪num_bits_list)
for num_bits in num_bits_list:
    avg_sd = results[num_bits]
    print(f"{num_bits}-bit quantization: Average SD = {avg_sd:.4f} dB")

print("Average SD for si747_8kHz.wav")
results, quantized_lpc = compute_sd_for_quantization_schemes(lpc_coeffs_female,␣
 ↪num_bits_list)
for num_bits in num_bits_list:
    avg_sd = results[num_bits]
    print(f"{num_bits}-bit quantization: Average SD = {avg_sd:.4f} dB")
```

```
Average SD for si745_8kHz.wav
5-bit quantization: Average SD = 0.9751 dB
4-bit quantization: Average SD = 2.0400 dB
3-bit quantization: Average SD = 3.1897 dB
Average SD for si747_8kHz.wav
5-bit quantization: Average SD = 0.9877 dB
4-bit quantization: Average SD = 2.1249 dB
3-bit quantization: Average SD = 3.6711 dB
```

[20]:

```python
def plot_lattice_coefficients(lpc_coef_unquant, frame_idx, num_bits=3):
    """
    Plot original and quantized lattice filter coefficients for one frame.

    Parameters
    ----------
    lpc_coef_unquant : 1D ndarray of length p
        Unquantized LPC coefficients for the selected frame
    frame_idx : int
        Frame index (for title)
    num_bits : int
        Number of bits for quantization (default: 3)
    """
    # Convert to reflection coefficients (original)
    k_original = lpc_to_reflection_coef(lpc_coef_unquant)

    # Quantize
    k_quantized = quantize_reflection_coefficients(k_original, num_bits)

    p = len(k_original)
    stages = np.arange(1, p + 1)

    # Create figure
    plt.figure(figsize=(6, 4))

    # Plot both original and quantized coefficients
    plt.plot(stages, k_original, 'b-o', label='Original', linewidth=2,␣
↪markersize=8)
    plt.plot(stages, k_quantized, 'r-s', label=f'{num_bits}-bit Quantized',␣
↪linewidth=2, markersize=8)


    # Formatting
    plt.xlabel('Lattice Stage (i)', fontsize=12)
    plt.ylabel('Reflection Coefficient k_i', fontsize=12)
    #plt.title(f'Lattice Filter Coefficients - Frame {frame_idx} - si745_8kHz.
↪wav', fontsize=14)
    plt.grid(True, alpha=0.3)
    plt.legend(fontsize=10)
    plt.ylim([-1.1, 1.1])
    plt.xticks(stages)

    plt.tight_layout()
    #plt.show()
    plt.savefig('../fig/lattice_filter_coef.png')
    plt.close()
```

[21]:
```
frame_idx = 11
plot_lattice_coefficients(lpc_coeffs_male[frame_idx, :], frame_idx)
```

```
<img src="../fig/lattice_filter_coef.png" alt="LPC Analysis Plot" style="width:650px; height:au
<p>
    <b>Figure 5.</b> Lattice filter reflection coefficients for frame 11 of the male speech sig
</p>
```

[22]:
```

```

```python
def lpc_analysis_quant(signal, fs, p, frame_length_ms, bits_lpc=3):
    """
    Perform LPC analysis on an entire signal (no frame overlap).

    Parameters
    ----------
    signal : 1D ndarray
        Input speech signal.
    fs : int
        Sampling frequency in Hz.
    p : int
        LPC order.
    frame_length_ms : float
        Frame length in milliseconds.
    bits_lpc: int
        number of bits used for lpc coefficients

    Returns
    -------
    residual_frames : 2D ndarray [num_frames, frame_length]
        LPC residual per frame.
    lpc_coeffs : 2D ndarray [num_frames, p]
        LPC coefficients per frame.
    lpc_coeffs_quant : 2D ndarray [num_frames, p]
        Quantized LPC coefficients per frame.
    """
    frame_matrix_orig, frame_matrix_windowed = windowing(signal,␣
↪frame_length_ms, fs)
    num_frames, _ = frame_matrix_windowed.shape

    residual_frames = np.zeros_like(frame_matrix_orig)
    lpc_coeffs = np.zeros((num_frames, p))
    lpc_coeffs_quant = np.zeros((num_frames, p))

    # Initial previous samples: zeros for the very first frame
    prev_samples = np.zeros(p)

    for i in range(num_frames):
        win_frame = frame_matrix_windowed[i, :]
        orig_frame = frame_matrix_orig[i, :]
        r = autocorrelation(win_frame, p)
        a = solve_lpc_coefficient(r)
        lpc_coeffs[i, :] = a

        reflection_coef = lpc_to_reflection_coef(a)
        reflection_coef_quant =␣
↪quantize_reflection_coefficients(reflection_coef, bits_lpc)
        lpc_coef_quant = reflection_to_lpc_coef(reflection_coef_quant)
        lpc_coeffs_quant[i, :] = lpc_coef_quant
        residual = lpc_residual_frame(orig_frame, lpc_coef_quant, prev_samples)
        residual_frames[i, :] = residual
```

[23]:
```
fs, male_speech = wav.read('../data/Male_speech/si745_8kHz.wav')
male_speech = male_speech / max(abs(male_speech))
display(Audio(male_speech, rate=fs))

residual_frames_male, lpc_coeffs_male, lpc_coeffs_quant_male =␣
 ↪lpc_analysis_quant(male_speech, fs, 10, 25)
residual_full_male = residual_frames_male.flatten()
residual_full_male = residual_full_male[:len(male_speech)]


t = np.arange(len(male_speech)) / fs
plt.figure(figsize=(6, 4))
plt.plot(t, male_speech, label='Original speech')
plt.plot(t, residual_full_male, label='LPC residual', alpha=0.8)

plt.xlabel('Time [s]')
plt.ylabel('Amplitude')
#plt.title('Original Speech Signal and LPC Residual with Quantized LPC coef -␣
 ↪si745_8kHz.wav')
plt.legend(loc='upper right')
plt.grid(True)
plt.tight_layout()
#plt.show()
plt.savefig('../fig/si745_resıdual.png')
plt.close()
```

```
<IPython.lib.display.Audio object>
```

```
<img src="../fig/si745_resıdual.png" alt="LPC Analysis Plot" style="width:650px; height:auto;"
<p>
    <b>Figure 6.</b> Original male speech waveform and LPC residual obtained using quantized LP
</p>
```

[24]:

```
fs, female_speech = wav.read('../data/Female_speech/si747_8kHz.wav')
female_speech = female_speech / max(abs(female_speech))
display(Audio(female_speech, rate=fs))

residual_frames_female, lpc_coeffs_female, lpc_coeffs_quant_female =␣
 ↪lpc_analysis_quant(female_speech, fs, 10, 25)
residual_full_female = residual_frames_female.flatten()
residual_full_female = residual_full_female[:len(female_speech)]

t = np.arange(len(female_speech)) / fs
plt.figure(figsize=(6, 4))
plt.plot(t, female_speech, label='Original speech')
plt.plot(t, residual_full_female, label='LPC residual', alpha=0.8)

plt.xlabel('Time [s]')
plt.ylabel('Amplitude')
#plt.title('Original Speech Signal and LPC Residual si747_8kHz.wav - quantized␣
 ↪LPC coefficients')
plt.legend(loc='upper right')
plt.grid(True)
plt.tight_layout()
#plt.show()
plt.savefig('../fig/si747_residual.png')
plt.close()
```

```
<IPython.lib.display.Audio object>
```

```
<img src="../fig/si747_residual.png" alt="LPC Analysis Plot" style="width:650px; height:auto;"
<p>
    <b>Figure 7.</b> Original female speech waveform and LPC residual obtained using quantized
</p>
```

[25]:

```
reconstruted_signal_male, reconstructed_frames_male =␣
 ↪lpc_synthesis(residual_frames_male, lpc_coeffs_quant_male)
display(Audio(reconstruted_signal_male, rate=fs))


t = np.arange(len(male_speech)) / fs
plt.figure(figsize=(6, 4))
plt.plot(t, male_speech, label='Original speech')
plt.plot(t, reconstruted_signal_male[:len(male_speech)], label='Reconstructed␣
 ↪speech', alpha=0.8)

plt.xlabel('Time [s]')
plt.ylabel('Amplitude')
#plt.title('Original Speech Signal and Reconstructed Speech with Quantized LPC␣
 ↪coef - si745_8kHz.wav')
plt.legend(loc='upper right')
plt.grid(True)
plt.tight_layout()
#plt.show()
plt.savefig('../fig/si745_qLPC_reconstruction.png')
plt.close()
```

```
<IPython.lib.display.Audio object>
```

```
<img src="../fig/si745_qLPC_reconstruction.png" alt="LPC Analysis Plot" style="width:650px; hei
<p>
    <b>Figure 8.</b>Comparison between the original male speech signal and the reconstructed sp
</p>
```

[26]:

```
reconstruted_signal_female, reconstructed_frames_female =␣
 ↪lpc_synthesis(residual_frames_female, lpc_coeffs_quant_female)
display(Audio(reconstruted_signal_female, rate=fs))

t = np.arange(len(female_speech)) / fs
plt.figure(figsize=(6, 4))
plt.plot(t, female_speech, label='Original speech')
plt.plot(t, reconstruted_signal_female[:len(female_speech)],␣
 ↪label='Reconstructed speech', alpha=0.8)

plt.xlabel('Time [s]')
plt.ylabel('Amplitude')
#plt.title('Original Speech Signal and Reconstructed Speech with Quantized LPC␣
 ↪coef - si747_8kHz.wav')
plt.legend(loc='upper right')
plt.grid(True)
plt.tight_layout()
#plt.show()
plt.savefig('../fig/si747_qLPC_reconstruction.png')
plt.close()
```

```
<IPython.lib.display.Audio object>
```

```
<img src="../fig/si747_qLPC_reconstruction.png" alt="LPC Analysis Plot" style="width:650px; he
<p>
    <b>Figure 9.</b>Comparison between the original female speech signal and the reconstructed
</p>
```

4 Codec

[27]:
```
def get_lowpass_filter():
    # Table 1 coefficients
    h = np.array([
        -0.016357422, -0.045654297, 0.0, 0.25073242, 0.70080566,
        1.0,
        0.70080566, 0.25073242, 0.0, -0.045654297, -0.016357422
    ])
    return h
```

[28]:
```
```

```python
def quantize_value_uniform(value, num_bits, min_val, max_val):
    levels = 2**num_bits
    step = (max_val - min_val) / levels

    val_clipped = np.clip(value, min_val, max_val)
    index = np.floor((val_clipped - min_val) / step)
    index = np.clip(index, 0, levels - 1)

    # Reconstruct
    quantized_val = (index + 0.5) * step + min_val
    return quantized_val
```

[29]:

```
from scipy.signal import filtfilt

class RPE_Codec:
    def __init__(self, frame_len=200, decimation_factor=3, grid_starts=4,␣
 ↪bits_gain=6, bits_residual=3):
        self.frame_len = frame_len
        self.h = get_lowpass_filter()
        self.grid_starts = grid_starts
        self.grid_start_indices = range(grid_starts)
        self.decimation_factor = decimation_factor
        self.samples_per_seq = frame_len // decimation_factor
        self.bits_grid = int(np.ceil(np.log2(self.grid_starts)))
        self.bits_gain = bits_gain
        self.bits_residual = bits_residual

    def encode_residual(self, residual_frame):
        """
        Encodes the residual using RPE (Downsampling + Quantization).
        Returns: quantized_subseq, grid_idx, quantized_gain
        """
        residual_lp = filtfilt(self.h, 1.0, residual_frame)

        best_energy = -1.0
        best_grid_idx = 0
        best_subseq = None

        for grid_idx in self.grid_start_indices:
            indices = np.arange(grid_idx, grid_idx + self.samples_per_seq *␣
 ↪self.decimation_factor, self.decimation_factor)
            subseq = residual_lp[indices]

            energy = np.sum(subseq**2)
            if energy > best_energy:
                best_energy = energy
                best_grid_idx = grid_idx
                best_subseq = subseq

        max_val = np.max(np.abs(best_subseq))

        normalized_subseq = best_subseq / max_val
        quantized_gain = quantize_value_uniform(max_val, num_bits=self.
 ↪bits_gain, min_val=0, max_val=1.0)
        quantized_subseq = quantize_value_uniform(normalized_subseq,␣
 ↪num_bits=self.bits_residual, min_val=-1.0, max_val=1.0)

        return quantized_subseq, best_grid_idx, quantized_gain

    def decode_residual(self, quantized_subseq, grid_idx, quantized_gain):
        """
        Reconstructs the full residual frame from RPE parameters.
        """
        reconstructed_subseq = quantized_subseq * quantized_gain
        full_residual = np.zeros(self.frame_len)
```

[30]:

```
class FullCodec:
    def __init__(self,
        sampling_rate,
        frame_length_ms,
        p,
        bits_lpc=3,
        decimation_factor=3,
        grid_starts=4,
        bits_gain=6,
        bits_residual=3
    ):
        self.sampling_rate = sampling_rate
        self.frame_length_ms = frame_length_ms
        self.frame_length = int(sampling_rate * frame_length_ms / 1000)
        self.p = p
        self.bits_lpc = bits_lpc
        self.rpe = RPE_Codec(self.frame_length, decimation_factor, grid_starts,␣
↪bits_gain, bits_residual)

    @property
    def bit_rate(self):
        total_bits_per_frame = self.bits_lpc * self.p + self.rpe.
↪calc_residual_total_bits_per_frame()
        bitrate_bps = total_bits_per_frame / self.frame_length_ms * 1000
        return bitrate_bps

    def encode_speech(self, signal):
        residual_frames, _, lpc_coeffs_quant  = lpc_analysis_quant(
            signal, self.sampling_rate, self.p, self.frame_length_ms, self.
↪bits_lpc)
        num_frames, _ = residual_frames.shape

        residual_frames_quant = []
        grid_idx_frames = np.zeros(num_frames, dtype=int)
        gain_frames_quant = np.zeros(num_frames)

        for frame_idx in range(num_frames):
            residual = residual_frames[frame_idx, :]
            residual_quant, grid_idx, gain_quant = self.rpe.
↪encode_residual(residual)
            residual_frames_quant.append(residual_quant)
            grid_idx_frames[frame_idx] = grid_idx
            gain_frames_quant[frame_idx] = gain_quant

        residual_frames_quant = np.array(residual_frames_quant)
        return lpc_coeffs_quant, residual_frames_quant, grid_idx_frames,␣
↪gain_frames_quant
```

```
    def decode_speech(self, lpc_coeffs_quant, residual_frames_quant,␣
↪grid_idx_frames, gain_frames_quant):
        num_frames, _ = residual_frames_quant.shape
        residual_frames_reconstructed = np.zeros((num_frames, self.
```

[31]:
```
def run_full_codec(signal, sampling_rate, p=10, frame_length_ms=25):
    codec = FullCodec(sampling_rate, frame_length_ms, p)
    lpc_coeffs_quant, residual_frames_quant, grid_idx_frames, gain_frames_quant␣
 ↪= codec.encode_speech(signal)
    reconstructed_signal, reconstructed_frames = codec.decode_speech(
        lpc_coeffs_quant, residual_frames_quant, grid_idx_frames,␣
 ↪gain_frames_quant)
    # Bitrate calculation
    bits_lpc = codec.p * codec.bits_lpc
    bits_grid = codec.rpe.bits_grid
    bits_gain = codec.rpe.bits_gain
    bits_samples = codec.rpe.samples_per_seq * codec.rpe.bits_residual
    total_bits_per_frame = bits_lpc + bits_grid + bits_gain + bits_samples

    results = {
        "reconstructed_signal": reconstructed_signal,
        "bitrate_bps": codec.bit_rate,
        "bit_allocation": {
            "total_per_frame": total_bits_per_frame,
            "lpc": bits_lpc,
            "grid_selection": bits_grid,
            "gain": bits_gain,
            "residual_samples": bits_samples
        },
        "fs": sampling_rate,
        "num_frames": reconstructed_frames.shape[0]
    }

    return results
```

Male & Female Examples

[32]:
```

```

```
fs, speech_signal = wav.read('../data/Male_speech/si745_8kHz.wav')
speech_signal = speech_signal / np.max(np.abs(speech_signal))

results = run_full_codec(speech_signal, fs, p=10, frame_length_ms=25)

reconstructed_speech = results['reconstructed_signal']
bitrate = results['bitrate_bps']

reconstructed_speech = reconstructed_speech[:len(speech_signal)]

print("Original Speech:")
display(Audio(speech_signal, rate=fs))

print(f"Coded Speech (Bitrate: {bitrate:.0f} bps):")
display(Audio(reconstructed_speech, rate=fs))

t = np.arange(len(speech_signal)) / fs

plt.figure(figsize=(6, 4))

plt.plot(t, speech_signal, label='Original Speech', color='blue', alpha=0.6)
plt.plot(t, reconstructed_speech, label='Coded Speech (Reconstructed)',␣
 ↪color='red', alpha=0.7, linestyle='--')

plt.xlabel('Time [s]')
plt.ylabel('Amplitude')
#plt.title(f'Codec Performance: Original vs. Coded Speech (~{bitrate/1000:.1f}␣
 ↪kbit/s)')
plt.legend(loc='upper right')
plt.grid(True, alpha=0.3)
plt.tight_layout()
#plt.show()
plt.savefig('../fig/si745_codec.png')
plt.close()
```

```
Original Speech:
```

```
<IPython.lib.display.Audio object>
```

```
Coded Speech (Bitrate: 9440 bps):
```

```
<IPython.lib.display.Audio object>
```

```
<img src="../fig/si745_codec.png" alt="LPC Analysis Plot" style="width:650px; height:auto;">
<p>
    <b>Figure 10.</b> Comparison between the original and coded male speech signal using the ir
</p>
```

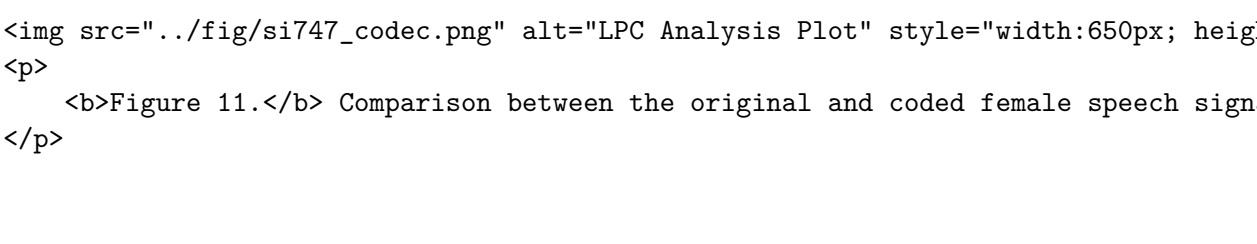
[33]:
```
fs, speech_signal = wav.read('../data/Female_speech/si747_8kHz.wav')
speech_signal = speech_signal / np.max(np.abs(speech_signal))

results = run_full_codec(speech_signal, fs, p=10, frame_length_ms=25)

reconstructed_speech = results['reconstructed_signal']
bitrate = results['bitrate_bps']

reconstructed_speech = reconstructed_speech[:len(speech_signal)]

print("Original Speech:")
display(Audio(speech_signal, rate=fs))

print(f"Coded Speech (Bitrate: {bitrate:.0f} bps):")
display(Audio(reconstructed_speech, rate=fs))

t = np.arange(len(speech_signal)) / fs

plt.figure(figsize=(6, 4))

plt.plot(t, speech_signal, label='Original Speech', color='blue', alpha=0.6)
plt.plot(t, reconstructed_speech, label='Coded Speech (Reconstructed)',␣
 ↪color='red', alpha=0.7, linestyle='--')

plt.xlabel('Time [s]')
plt.ylabel('Amplitude')
#plt.title(f'Codec Performance: Original vs. Coded Speech (~{bitrate/1000:.1f}␣
 ↪kbit/s)')
plt.legend(loc='upper right')
plt.grid(True, alpha=0.3)
plt.tight_layout()
#plt.show()
plt.savefig('../fig/si747_codec.png')
plt.close()
```

```
Original Speech:
```

```
<IPython.lib.display.Audio object>
```

```
Coded Speech (Bitrate: 9440 bps):
```

```
<IPython.lib.display.Audio object>
```



```
<p>
    <b>Figure 11.</b> Comparison between the original and coded female speech signal using the
</p>
```

[34]:
```
import os
import glob

data_folder = '../data/'
output_base_folder = '../results/coded_signals/'

categories = ['Male_speech', 'Female_speech']

for category in categories:
    input_dir = os.path.join(data_folder, category)
    output_dir = os.path.join(output_base_folder, category)

    if not os.path.exists(output_dir):
        os.makedirs(output_dir)

    search_pattern = os.path.join(input_dir, '*.wav')
    wav_files = glob.glob(search_pattern)

    if not wav_files:
        print(f"Warning: No .wav files found in '{input_dir}'")
        continue

    print(f"\nProcessing {len(wav_files)} files in '{category}' folder...")

    for file_path in wav_files:
        filename = os.path.basename(file_path)

        fs, signal = wav.read(file_path)
        if signal.dtype == np.int16:
            signal = signal / 32768.0

        results = run_full_codec(signal, fs)

        output_path = os.path.join(output_dir, 'coded_' + filename)
        reconstructed_signal = results['reconstructed_signal']

        reconstructed_signal = np.clip(reconstructed_signal, -1.0, 1.0)

        wav.write(output_path, fs, (reconstructed_signal * 32767).astype(np.
 ↪int16))

        print(f"  Saved: {category}/coded_{filename} | Bitrate =␣
 ↪{results['bitrate_bps']:.0f} bps")
```

```
Processing 10 files in 'Male_speech' folder…

  Saved: Male_speech/coded_si745_8kHz.wav | Bitrate = 9440 bps
  Saved: Male_speech/coded_si1899_8kHz.wav | Bitrate = 9440 bps
  Saved: Male_speech/coded_si1175_8kHz.wav | Bitrate = 9440 bps
  Saved: Male_speech/coded_si1230_8kHz.wav | Bitrate = 9440 bps
  Saved: Male_speech/coded_si923_8kHz.wav | Bitrate = 9440 bps
  Saved: Male_speech/coded_si1039_8kHz.wav | Bitrate = 9440 bps
```

4.0.1 Subjective evaluation

[35]:
```
def play_dcr_pair(original, coded, fs):
    silence = np.zeros(int(0.5 * fs))
    sample = np.concatenate([original, silence, coded])
    display(Audio(sample, rate=fs))
```

[36]:
```
for category in categories:
    input_dir = os.path.join(data_folder, category)
    output_dir = os.path.join(output_base_folder, category)

    wav_files = glob.glob(os.path.join(input_dir, '*.wav'))

    print(f"\n### Comparison for {category} ### \n")

    for original_path in wav_files:
        filename = os.path.basename(original_path)
        coded_path = os.path.join(output_dir, 'coded_' + filename)
        if os.path.exists(coded_path):
            fs_orig, original_sig = wav.read(original_path)
            fs_coded, coded_sig = wav.read(coded_path)

            print(f"Playing: {filename}")
            play_dcr_pair(original_sig, coded_sig, fs_orig)
```

```
### Comparison for Male_speech ###

Playing: si745_8kHz.wav
<IPython.lib.display.Audio object>
Playing: si1899_8kHz.wav
<IPython.lib.display.Audio object>
Playing: si1175_8kHz.wav
<IPython.lib.display.Audio object>
Playing: si1230_8kHz.wav
<IPython.lib.display.Audio object>
Playing: si923_8kHz.wav
<IPython.lib.display.Audio object>
Playing: si1039_8kHz.wav
<IPython.lib.display.Audio object>
Playing: si1364_8kHz.wav
<IPython.lib.display.Audio object>
Playing: si1010_8kHz.wav
<IPython.lib.display.Audio object>
Playing: si1024_8kHz.wav
<IPython.lib.display.Audio object>
Playing: si839_8kHz.wav
<IPython.lib.display.Audio object>

### Comparison for Female_speech ###
```